

# Deadlock Analysis Report: deadlock.py

---

## Overview

This report documents the deadlock scenario implemented in `deadlock.py`. The program demonstrates a classic deadlock condition in a multi-threaded environment using mutual exclusion locks (mutexes).

---

## Problem Description

### What is Deadlock?

A deadlock occurs when two or more threads are blocked forever, waiting for each other to release resources. The program cannot proceed because each thread holds a resource that another thread needs.

### Deadlock Conditions

All four of the following conditions must be true for deadlock to occur:

1. **Mutual Exclusion:** Resources cannot be shared (only one thread can hold a lock)
  2. **Hold and Wait:** Threads hold resources while waiting for others
  3. **No Preemption:** Resources cannot be forcibly taken away
  4. **Circular Wait:** A circular chain of threads waiting for resources
- 

## Code Structure

### 1. Account Class

```
class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.lock = threading.Lock() # Mutex for this resource
```

- Represents a bank account with a unique name, balance, and lock
- Each account has its own `threading.Lock()` to protect concurrent access

### 2. Transfer Function (Problematic Version)

The code contains two versions of the `transfer()` function:

#### Version 1 (Commented Out - Correct):

```
def transfer(source, target, amount):
    first, second = (source, target) if source.name < target.name else
    (target, source)
```

```
with first.lock:
    with second.lock:
        source.balance -= amount
        target.balance += amount
```

- Orders locks alphabetically to prevent deadlock
- Always acquires locks in the same order

Version 2 (Active - Problematic):

```
def transfer(source, target, amount):
    with source.lock:
        print(f" [LOCKED] {source.name}...")
        time.sleep(1) # Artificial delay
    with target.lock:
        source.balance -= amount
        target.balance += amount
```

- Acquires locks in the order: source → target
- Contains intentional delay to increase deadlock probability

3. Scenario Setup

```
acc1 = Account("Account_1", 1000)
acc2 = Account("Account_2", 1000)

t1 = threading.Thread(target=transfer, args=(acc1, acc2, 100),
name="Thread-A")
t2 = threading.Thread(target=transfer, args=(acc2, acc1, 50), name="Thread-
B")
```

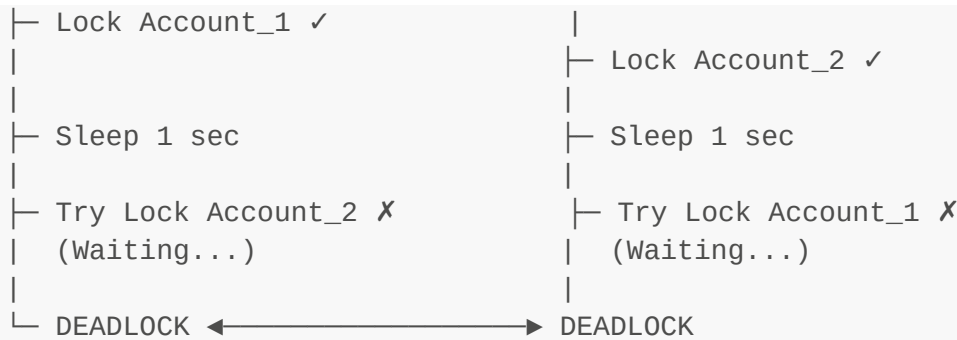
Deadlock Sequence

Timeline of Events

1. **Thread-A starts:** Locks Account\_1
2. **Thread-B starts:** Locks Account\_2
3. **Thread-A waits:** Tries to lock Account\_2 (already held by Thread-B) → BLOCKED
4. **Thread-B waits:** Tries to lock Account\_1 (already held by Thread-A) → BLOCKED
5. **Deadlock:** Both threads are stuck indefinitely

Visual Representation





## Why Deadlock Occurs

### Circular Wait Chain

- **Thread-A:** Holds `Account_1.lock`, wants `Account_2.lock`
- **Thread-B:** Holds `Account_2.lock`, wants `Account_1.lock`
- Neither thread can proceed because the resource it needs is held by the other

### Artificial Delay

The `time.sleep(1)` in the transfer function increases the probability of both threads acquiring their first lock before either attempts to acquire the second lock.

## Deadlock Prevention Strategy

### Solution: Lock Ordering

The commented-out version demonstrates the fix:

```

def transfer(source, target, amount):
    # Always lock in the same order (by name)
    first, second = (source, target) if source.name < target.name else
    (target, source)

    with first.lock:
        with second.lock:
            source.balance -= amount
            target.balance += amount
  
```

### Why This Works:

- **Breaks Circular Wait:** By always acquiring locks in alphabetical order, no circular dependency can form
- Both `transfer(acc1, acc2, 100)` and `transfer(acc2, acc1, 50)` acquire locks in the order: `Account_1` → `Account_2`
- One thread will acquire the first lock while the other waits, preventing circular waiting

# Deadlock Detection

## Program Behavior

- **Expected Output:** "This line will never be reached."
- **Actual Output:** The program freezes indefinitely without printing this message
- **Symptom:** High CPU usage or complete system freeze (depending on threading implementation)

## How to Detect

The absence of the final print statement indicates deadlock. In a real system:

- Monitor thread states (use `threading.enumerate()`)
- Set timeouts on locks
- Use deadlock detection algorithms

---

## Key Takeaways

| Aspect             | Details                             |
|--------------------|-------------------------------------|
| Problem Type       | Circular Wait Deadlock              |
| Root Cause         | Inconsistent lock acquisition order |
| Prevention         | Enforce global lock ordering        |
| Timeout Prevention | Set acquisition timeouts            |
| Detection          | Monitor for blocked threads         |

---

## References

- **Deadlock Prevention Techniques:** Lock ordering, resource allocation graphs
  - **Threading in Python:** `threading.Lock()`, context managers (`with` statement)
  - **Real-world Application:** Database transactions, file systems, OS resource management
- 

## Conclusion

This program successfully demonstrates how deadlock occurs in multi-threaded systems. By uncommenting the corrected `transfer()` function and commenting out the problematic version, the deadlock can be prevented through proper lock ordering, illustrating a fundamental principle in concurrent programming.